

삼성청년 SW·AI아카데미

B형 특강

<알림>

본 강의는 삼성청년SW·AI아카데미의 콘텐츠로
보안서약서에 의거하여
강의 내용을 어떠한 사유로도 임의로 복사, 촬영,
녹음, 복제, 보관, 전송하거나
허가 받지 않은 저장매체를
이용한 보관, 제3자에게 누설, 공개,
또는 사용하는 등의 행위를 금합니다.

Pro 연습 문제. 동굴 퍼즐

동굴 퍼즐 - 문제 이해

전국의 몬스터를 모두 잡기 위해 모험을 떠난 소년이 있다.

이 소년은 모험을 하던 중, 퍼즐을 풀어야만 나갈 수 있는 $N \times N$ 크기의 동굴에 들어가게 되었다. ($5 \leq N \leq 30$)

이 동굴을 탈출하려면 **최소한의 움직임**으로 바위를 밀어 동굴 어딘가에 존재하는 구멍에 집어 넣어야 한다.

아래는 10×10 크기의 동굴의 예시이다. ($N=10$)

그리고 주요 요소들의 위치는 [Fig.1]과 같다.

- 소년 : (4, 8)
- 바위 : (4, 7)
- 구멍 : (2, 8)

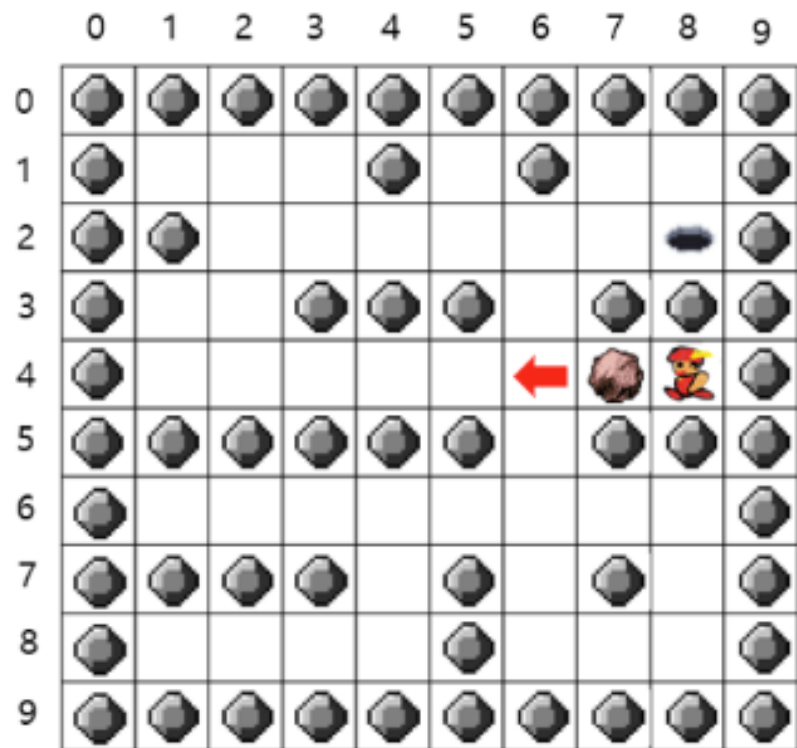
	0	1	2	3	4	5	6	7	8	9
0										
1										
2										
3										
4										
5										
6										
7										
8										
9										

[Fig. 1]

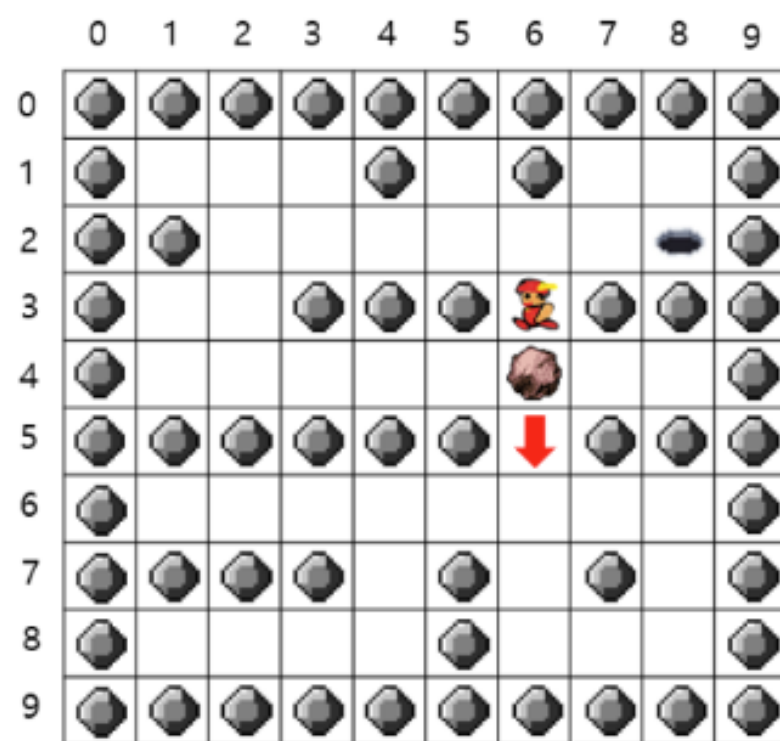
동굴 퍼즐 - 문제 이해

소년이 바위를 특정 방향으로 밀기 위해서는, 그 방향의 반대편에 위치하고 있어야 한다.

[Fig.2]에서는 소년이 바위를 왼쪽으로 밀 수 있고, [Fig.3]에서는 바위를 아래 방향으로 밀 수 있다.



[Fig. 2]



[Fig. 3]

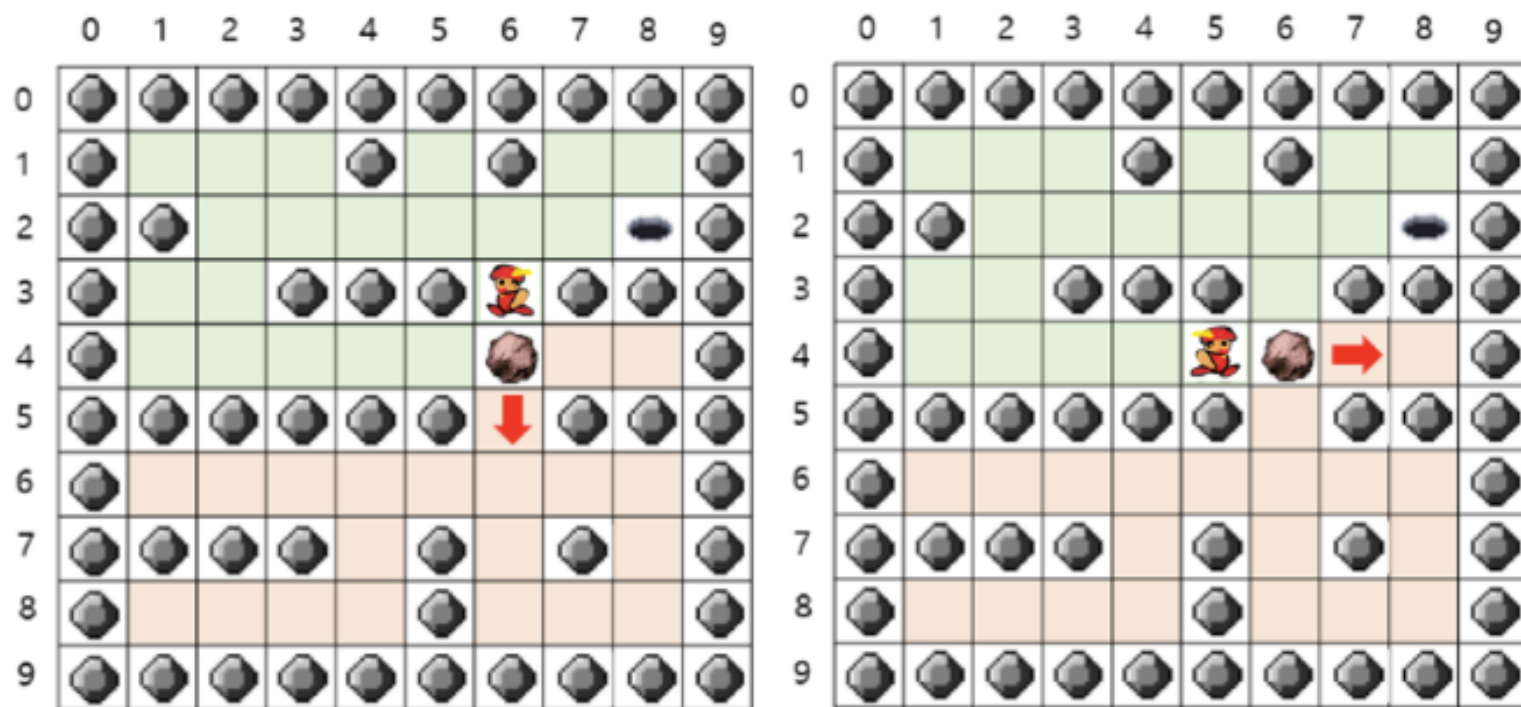
동굴 퍼즐 - 문제 이해

그렇기 때문에 특정 방향으로 바위를 밀기 위해서는, 소년이 바위를 밀 수 있는 위치로 이동이 가능하여야 한다.

[Fig.4]와 같이 바위가 (4,6) 위치에 놓인 상황에서는 소년이 바위를 아래와 오른쪽 방향으로 밀 수 있지만, 다른 방향으로는 밀 수 없다.

초록색으로 색칠된 공간은 소년이 현재 위치에서 이동 가능한 공간, 그리고 빨간색으로 색칠된 공간은 소년이 이동할 수 없는 공간을 나타낸다.

현재 돌이 존재하는 위치로는 소년이 이동할 수 없다. (구멍이 존재하는 칸은 이동이 가능하다)



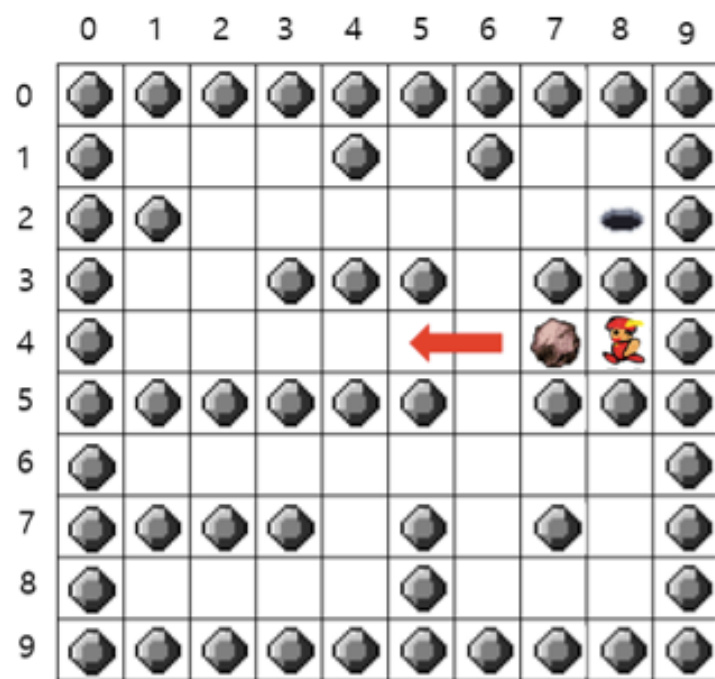
[Fig. 4]

동굴 퍼즐 - 문제 이해

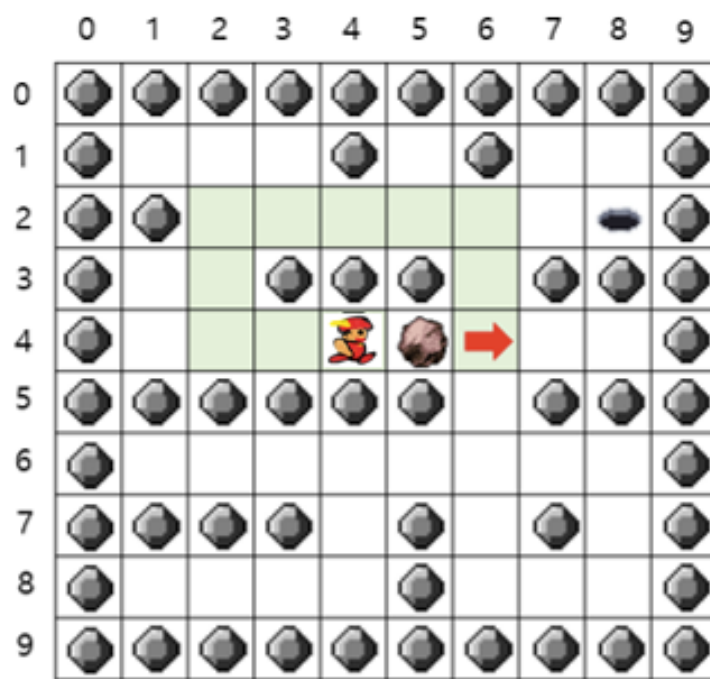
현재 예시에서는 [Fig.5]와 같은 경로를 통해 13번의 이동으로 바위를 구멍에 넣을 수 있다.

아래 예시에서 빨간 화살표는 바위가 이동하는 방향과 거리, 초록색으로 칠해진 칸은 소년이 바위를 밀기 위해 이동한 경로를 나타낸다.

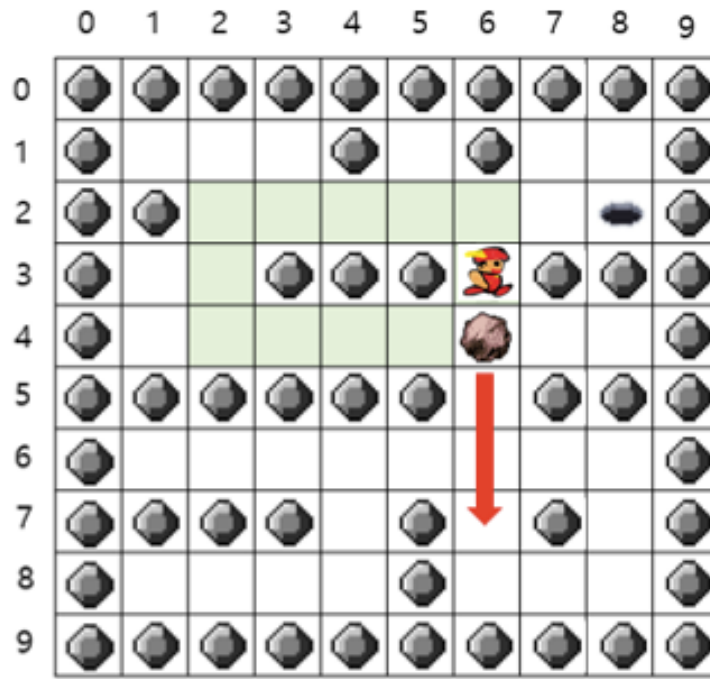
이동 횟수는 바위가 이동한 횟수만을 의미한다. (소년이 바위를 특정 방향으로 밀기 위해 이동한 횟수는 고려하지 않는다.)



총 이동 횟수 : 2

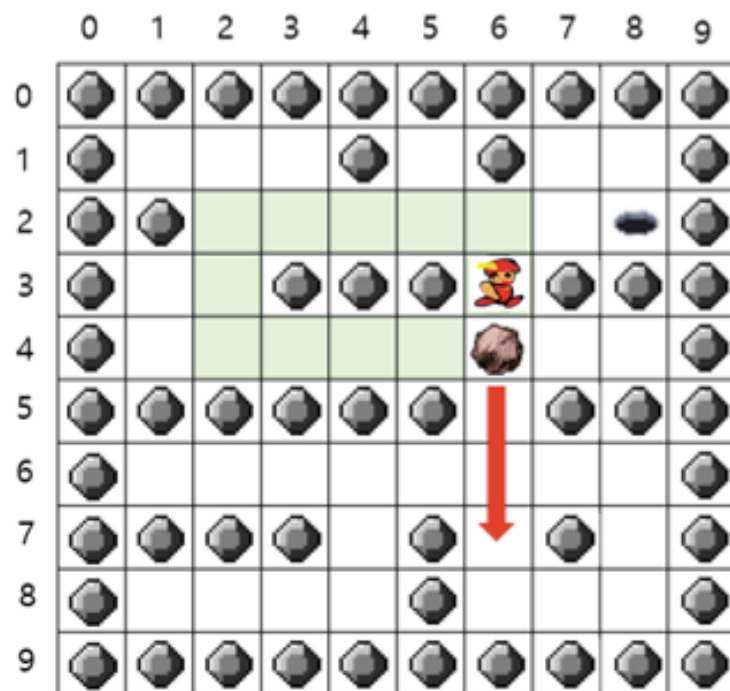


총 이동 횟수 : 3

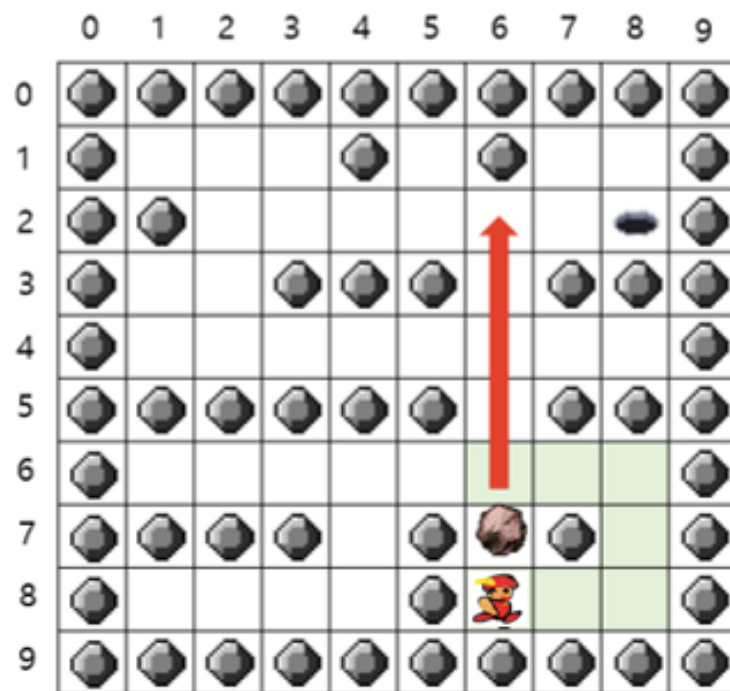


총 이동 횟수 : 6

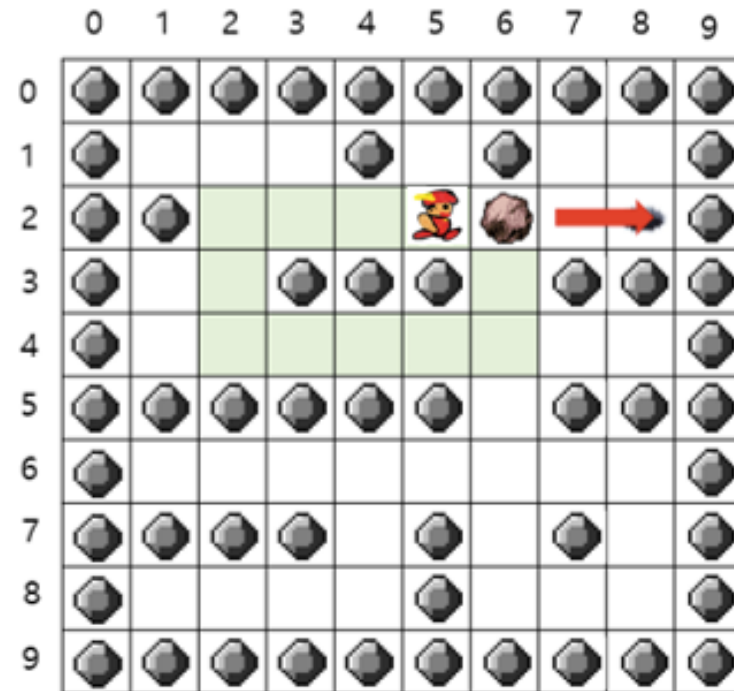
동굴 퍼즐 - 문제 이해



총 이동 횟수 : 6



총 이동 횟수 : 11



총 이동 횟수 : 13

[Fig. 5]

동굴 퍼즐 - API

```
def init(N:int, mMap:List[List[int]]) -> None:
```

각 테스트케이스의 처음에 호출된다.

- **[Fig. 7]** 과 같이 행, 열 좌표로 구역을 나타낼 때, 구역 (Row, Col) 의 지형 정보는 mMap[Row][Col]에 저장되어 있다.
- mMap[Row][Col]의 값이 0이면 캐릭터와 돌이 이동 가능한 구역을, 1이면 이동 불가능한 구역을 의미한다.

	0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0	0
1	0	1	1	1	0	1	1	1	1	0
2	0	0	1	1	1	1	1	1	1	0
3	0	1	1	0	0	0	1	0	0	0
4	0	1	1	1	1	1	1	1	1	0
5	0	0	0	0	0	1	1	0	0	0
6	0	1	1	1	1	1	1	1	1	0
7	0	0	0	0	1	1	1	1	1	0
8	0	1	1	1	1	1	1	1	1	0
9	0	0	0	0	0	0	0	0	0	0

[Fig. 7]

Parameters

- N : 지도의 크기 N x N ($10 \leq N \leq 30$)
- mMap[] : 각 구역의 이동 가능 여부. ($0 \leq \text{mMap}[] \leq 1$)

동굴 퍼즐 - API

```
def push(mRockR:int, mRockC:int, mDir:int, mGoalR:int, mGoalC:int) -> int:
```

바위, 소년, 구멍의 위치가 주어지며, 구멍까지 바위를 미는 최소 횟수를 반환한다.

- 시작 시 소년은 바위와 이웃해 있으며, 바위에 대한 상대적 위치가 mDir으로 주어진다.
- mDir 값에 따른 소년의 위치는 [Fig. 8] 과 같다.



[Fig. 8]

Parameters

- mRockR, mRockC : 바위의 행, 열 좌표 ($0 \leq mRockR, mRockC < N$)
- mDir : 바위와 소년의 상대적 위치 ($0 \leq mDir \leq 3$)
- mGoalR, mGoalC : 구멍의 행, 열 좌표 ($0 \leq mGoalR, mGoalC < N$)

Return

- 바위를 최소한으로 밀면서 구멍까지 이동시킬 때, 바위를 미는 횟수

- 바위의 시작 위치와 목표 지점은 항상 다르다.
- 바위와 소년의 시작 위치는 항상 이동 가능한 구역이다.
- 구멍까지 바위를 이동할 수 없는 경우는 주어지지 않는다.
- 바위를 미는 횟수는 최소가 되어야 하나, 캐릭터의 이동 횟수는 무관하다.

[제약사항]

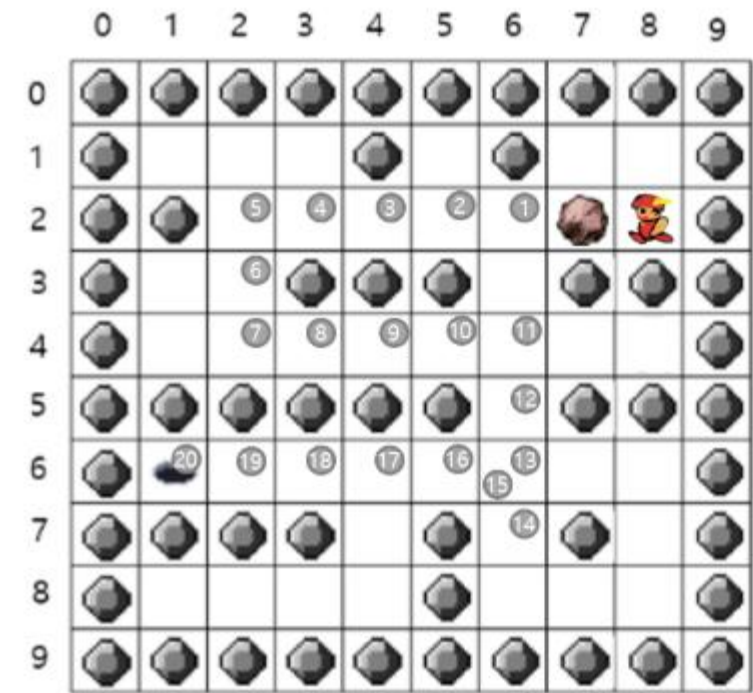
1. 각 테스트 케이스 시작 시 init() 함수가 호출된다.
2. 주어지는 지도는 이동 불가능한 구역으로 둘러 쌓여있다.
3. 각 테스트 케이스에서 push() 함수의 호출 횟수는 10 이하이다.

동굴 퍼즐 - 예시

[예제]
아래의 예제는 TC 1번의 내용을 기술한다.

Order	Function	Return	Figure
1	init(10, {{1, 1, 1, 1, 1, 1, ...}, {1, 0, 0, 0, 1, ...}, ...})		
2	push(2, 7, 1, 6, 1)	20	Fig. 6
3	push(4, 7, 1, 2, 8)	13	Fig. 5

(순서 2) [Fig.6]의 과정으로 20회 밀면 구멍에 도달한다.



[Fig. 6]

(순서 3) 본문 [Fig.5]의 과정으로 13회 밀면 구멍에 도달한다.

설계하기

삼성청년SW·AI아카데미

❖ BFS

- BFS 문제임을 파악할 수 있는 요소들
 - 1) 무가중치 그래프
 - 2) 시뮬레이션 필요
 - 3) 최소 이동 (최단 거리)

❖ 시뮬레이션 문제

- API가 단 한 개 존재한다. (init() 제외)
- 익숙한 A형급 문제

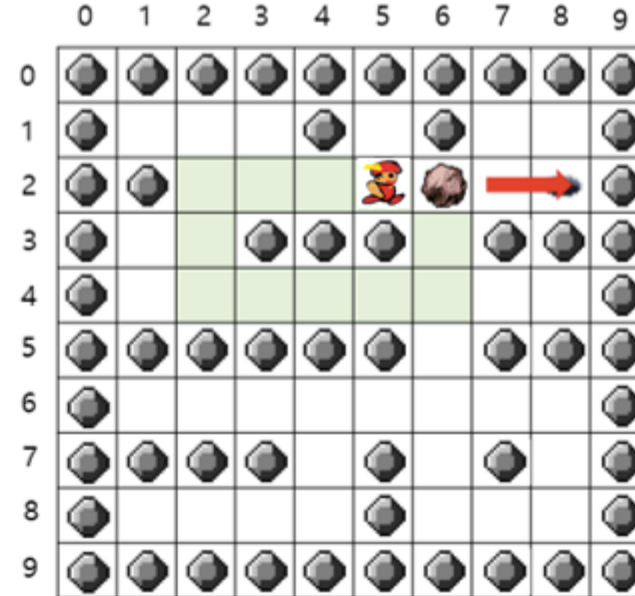
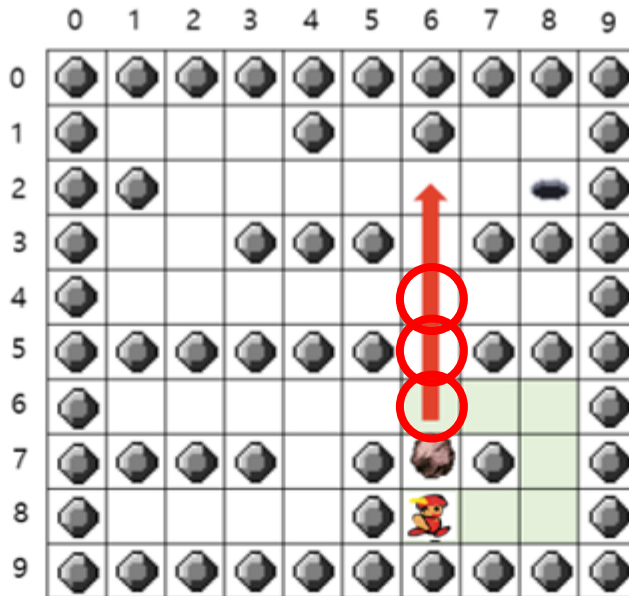
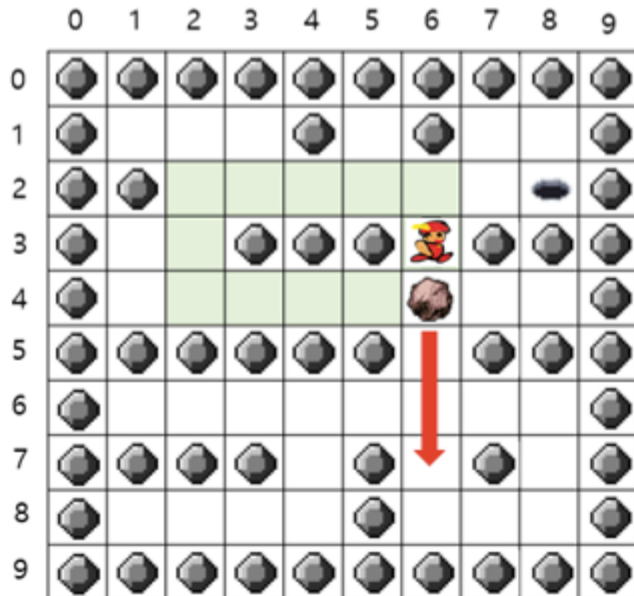
push() 계산

❖ 맵 크기 : 최대 30 x 30

- BFS / Flood Fill 채용시 $O(N^2) = 900$

❖ 문제점

- 소년이 바위를 밀어줘야 함 (BFS의 이동 조건 추가)
- 소년과 바위는 왔던곳을 계속 왔다갔다 가능 → BFS의 시간복잡도가 성립되지 않음
- 큐에서 memory overflow가 발생할 수도 있음

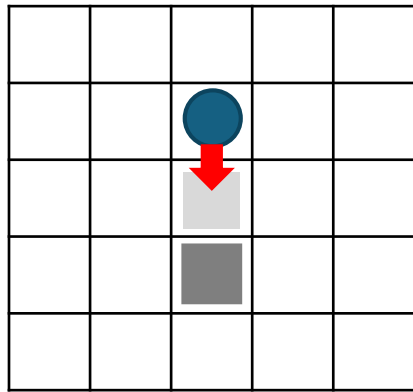


❖ 모든 칸을 최대 몇 번이나 이동할 수 있을까?

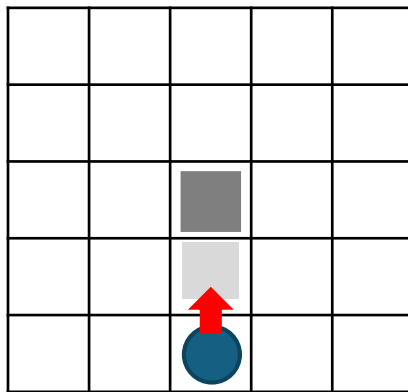
- 계속 왔다갔다 하면 무한대로 이동 가능
- 메모리만 터지지 않으면 불필요한 가능성을 엄청 많이 보겠지만 해결은 가능
- 단, 시간은 보장하지 못함

❖ 다음 경우를 생각해보자.

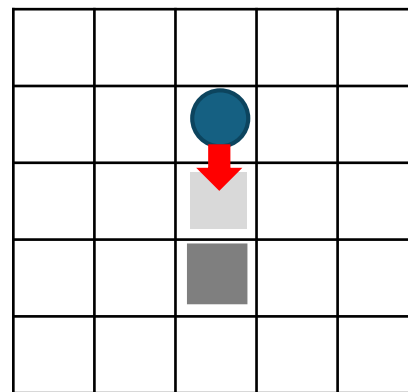
- 아래와 같은 상황이 발생할까? (YES / NO)
- NO라면, 그렇다면 특정 칸은 아무리 많아봐야 몇 번 방문하게 될까?



바위를 아래로 밀었다



바위를 위로 밀었다



바위를 다시 아래로 밀었다

push() 계산 (re)

❖ 목표는 “최소 이동”

- 바위를 다시 같은 방향에서, 같은 위치로 밀게 될 일은 없다. → 불필요한 이동
- 즉, 한 칸은 최대 방향의 개수, 즉, 4 방향에서 각각 한 번씩 방문이 가능하다.

❖ 맵 크기 : 최대 30 x 30

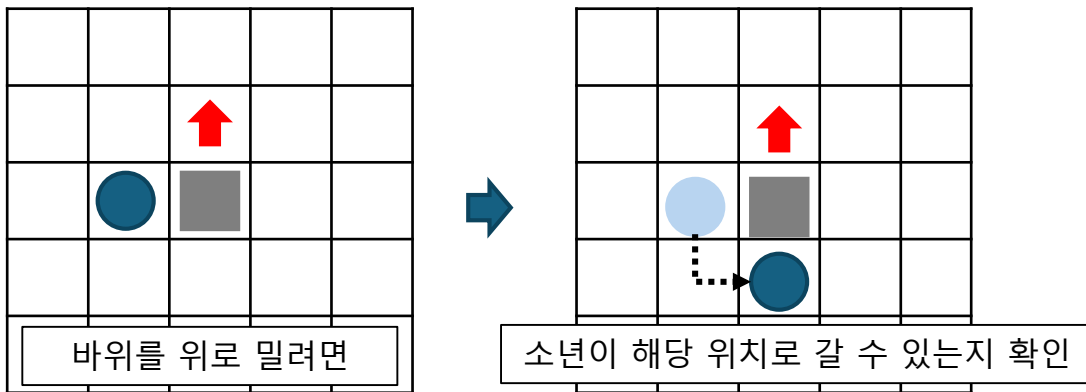
- BFS / Flood Fill 채용시 $O(N^2) = 900$
- 각 칸을 최대 4번씩 방문 가능 = $900 \times 4 = \underline{3,600}$
- + 소년이 이동하는 BFS 등

❖ push()

- 바위의 위치, 소년의 상대적 위치, 구멍의 위치가 주어진다.
 - 소년의 정확한 좌표를 찾기 위해서는 방향배열을 사용한다. (상우하좌)
- 바위의 위치를 기반으로 bfs를 수행한다.
 - 바위가 특정 위치에 있을 때, 소년이 어디 있는지도 알아야 한다.

```
class Node {  
    int rockY, int rockX, int boyY, int boyX;  
}
```

- 바위를 다음 위치(ny, nx)로 이동시키려 할 때,
소년이 바위를 그쪽으로 밀기 위한 위치로 이동이 가능한지 확인한다.



- 특정 위치에 바위가 "이쪽 방향에서 온적이 있음"을 기록한다.
- 구멍에 도착할때까지 (실패하는 경우는 없음) 수행한다.

최종 시간복잡도

❖ 시간복잡도

- 바위의 bfs :
 - 모든 칸 4번씩 방문 가능
 - $O(4 \times (N \times N)) = 3,600$
- 소년의 bfs :
 - 바위가 특정 위치에서 4방향으로 가려고 할때 1번의 bfs
 - $3,600 \times O(N \times N) = 3,240,000$
- testcase의 개수
 - $10 \times 3,240,000 = 32,400,000$